The Log Collection System

instmon keeps two logs. Its activity log is the short list of what instmon has just done. The Suite Log is a timed collection, started by hand, that gathers journal lines, file lines, kernel lines and health snapshots from the whole suite into one file, with secrets hidden. This summary covers both, the sources, the limits, the redaction, and how the collection starts, stops and is read.

ACTIVITY LOG | SUITE LOG | REDACTION | LIMITS

SUMMARY · READ-ONLY REVIEW

The activity log and the Suite Log

	ACTIVITY LOG	SUITE LOG
What it holds	What instmon has just done: installs, checks, job results	Lines from the whole suite: journals, log files, kernel messages, Wi-Fi state, periodic health snapshots
Where it lives	In memory, the last 500 lines	A file in `/run/asl_dvs`, rotated at 10 MB, with one older copy
Survives a restart?	No	The file survives an instmon restart, and the collection resumes if it was running (Part 4). It does not survive a reboot, because `/run` is cleared
How it is read	The `/api/log` route returns the lines after an id	The Suite log viewer at `/suitelog`, and `/api/suitelog/*`
Started by	Always on	The Start button, for a set number of hours

The activity log is what the Log card on the page shows. It is a short working record. The Suite Log is for after-the-fact questions: what happened across several tools, in a time window.

Where the Suite Log reads from

2.1 Journal units

The collector follows the systemd journal for a list of units: the Node Talker, Node Listen, Node Recorder, Pibrary, wifimon, instmon, the Dashboard, SysMon, 44helper and its netready unit, the Zello bridge, the radio services (the analog and MMDVM bridges, Asterisk, the SVXLink USRP service, the STFU service, the M17 service), Allmon3, the WireGuard unit, and the Amp services. Units named with a pattern (for example `asl_dvs_*` and `wg-quick@*`) are matched when they run.

Besides the fixed list, any other service file in `/etc/systemd/system` whose first 4 KB mentions asl_dvs, ASL-DVS, DVSwitch, Node Talker, Node Recorder or Node Control is added. The journal is read with `journalctl -f -n 0 -o json` at low priority (nice 10) from the moment of start, so only new lines are collected. The unit list is rechecked every 5 minutes, and the reader is restarted when the list changes or the reader stops.

2.2 Kernel messages

A second journal reader follows the kernel log (`journalctl -k`). Lines of warning level and above are kept whatever they say. Ordinary (info) lines are kept only when they match a fixed set of patterns: Wi-Fi chip and interface messages, MMC, USB, under-voltage and throttling, thermal, out-of-memory, voltage, firmware, I/O errors, ext4 filesystem messages and resets. Debug lines are dropped. The kernel lines carry a level, so the urgent ones are never dropped by the rate limit (Part 3.3).

2.3 Log files

The file list is rechecked every 30 seconds, and files ending in `.gz`, `.1` or `.old` are left out. Files are read by tail: a new file is read from its last 50 lines (taken from its final 64 KB), then followed, up to 1 MB each second. A file that is replaced or shortened is read again from the start. A line is marked as an error if it contains "error", "critical" or "fatal", as a warning if it contains "warn", and as information otherwise. The list covers Asterisk's full and messages logs, the MMDVM, USRP, DVSwitch and ircDDB gateway logs, the wifimon logs, the SVXLink log, the instmon full-update job log, and the Pibrary logs found on any mounted drive.

2.4 Wi-Fi state and snapshots

- **Wi-Fi.** Every 10 seconds the link state file `/run/asl_dvs/wifi_link.json` (written by wifimon) is read, and a row is written with the link flag, band, channel, signal and noise, bit rates, packet, retry and failure counts, beacon loss, connected seconds, blips, drops and chip stalls.
- **Snapshots.** Every 10 minutes the collector writes a snapshot row with the load averages, the memory and swap figures from `/proc`, the free and total space of `/run` and `/`, the CPU temperature, the Pi's throttle flags (from `vcgencmd get_throttled`, when present), and the active state of each unit in the list.

What each row looks like and what limits it

3.1 A row

Each row is one line of JSON with a time in milliseconds (`t`), a source (`src`, for example `journal:nodecontrol` or `file:…`), a level (`lvl`: debug, info, warn or err; for journal lines it comes from the journal priority, with 3 and below as err, 4 as warn, 5 and 6 as info, 7 as debug), a kind (`kind`: for example log, mark, drop, snap, wifi, the receive-health rows `rxh` that Node Talker writes, or a kerchunk kind), a message, and optional structured data. Lines are cut to 2,000 characters, and control characters are removed.

3.2 Repeats are collapsed

When the same message arrives again from the same source within 60 seconds, it is counted, not written again. When the run ends, the count is written as one row. The comparison ignores the timestamp at the start of a line, so the same message with a new time still counts as a repeat.

3.3 Rate limit per source

RULE	VALUE
Bytes per source per minute	98,304 (96 KB)
Lines over the budget	Dropped and counted. A warning row records how many were dropped in the minute.
Warnings and errors	Never dropped by the rate limit (urgent).

The total dropped is kept in the collection's counters.

3.4 Query limits

A query returns at most 300 rows by default, newest first, and at most 5,000 rows in any case. A query can filter by time window, source, level, kind and text.

The collection's life

1. **Start.** The Start button asks for a number of hours, from 0 to 168 (default 12). Zero means it runs until stopped. A random salt is made for this session and written to the flag file, and the end time to a second file next to it (`.until`). The start is logged in the file as a mark row whose data records the instmon version, the kernel, whether the journal is on disk or in memory, the node's uptime, the file list, the unit list and the hours.
2. **Collect.** The readers run as threads: the unit journal reader, the kernel reader, the file reader, the Wi-Fi reader, the memory guard and the snapshot writer.
3. **Stop.** Stop kills the reader processes, writes any repeats still held, writes a stop mark with the reason and the counts, and removes the flag file. The salt is cleared.
4. **Automatic stops.** The collection stops by itself when its time is up, or when free memory drops below 60 MB (61,440 KB). Both are checked every 30 seconds. The reason is written in the stop mark.
5. **Resume.** If instmon restarts while the flag file exists, the collection starts again with the same salt, its end time read from the saved "until" file, and a mark row "collection resumed after an instmon restart".

4.1 The flag file

The flag file `/run/asl_dvs/suitelog.on` holds the salt. It lives in `/run`, so it is gone after a reboot, and the collection does not resume after one. Stopping removes it.

Hiding secrets and tagging addresses

Every row is redacted before it is written, when a salt is set (that is, while a collection runs). The redaction is applied to the whole row after it is turned into JSON, so it also covers the structured data. The same function is used by Node Talker, so both tools' rows are treated the same way.

WHAT	WHAT IS WRITTEN INSTEAD
Bearer tokens	`Bearer [hidden]`
Passwords, passphrases, secrets, tokens, API keys, auth and cookie values (after a word and a separator)	`[hidden]` in place of the value
A quoted network name after the word ssid	A tag such as `ssid:` followed by four hex characters. An SSID that is not quoted is not matched
MAC addresses	A `mac:` tag with four hex characters
IPv4 and IPv6 addresses, including LAN addresses (all except loopback and unspecified)	An `ip:` tag with four hex characters
Loopback and unspecified addresses	Kept as they are

Each tag is the first four hex characters of a SHA-256 hash of the salt and the value, in lower case. The salt is random for each collection. So the same SSID or address always gets the same tag within one collection, which keeps the rows linkable, but a tag cannot be reversed to the value, and it is not the same across collections.

The rows written by a collection are redacted, but the file is only as safe as its readers. The book should say that the collection is meant for sharing with a helper, after a check of its contents.

5.1 Kerchunk rows

Rows whose kind begins `kc` (`kc`, `kc_label`, `kc_reset`, `kc_base`) are not kernel messages. They are Node Talker's kerchunk log (RX path, Part 2.2), which Node Talker stores in this same file. Node Talker writes them whether or not a collection is running, and without redaction, because the salt is only set during a collection. Clear keeps them, so the log is not lost when the collection is cleared. The file is in `/run`, so a reboot ends them too.

The viewer and its controls

- **Query.** The viewer filters rows by time window, source, level, kind and text (Part 3.4).
- **Sources.** The list of sources that have rows, with their counts and last times, can be refreshed by the operator (`force`).
- **Coverage.** A coverage report lists each expected unit and file, and whether it has produced rows. A unit that is not running is shown as such. A collection that has run for at least 10 minutes is judged on that basis.
- **Download.** The whole file, the old rotated copy included, downloads as `suitelog-<time>.jsonl`. A view can also be downloaded as text or as JSON lines.
- **Clear.** Clear removes the file and its old copy, and writes back only the kerchunk rows (Part 5.1). The counters are reset.

6.1 Rotation

When the file reaches 10 MB, it is moved to a name ending in `.1`, replacing any older copy. The two files together are the collection. Rows are read from the old copy first, then the new one.

instmon's own short log

Every message instmon logs goes into an in-memory ring of 500 lines, with an increasing id. The Log card asks for lines after the last id it has seen, so it gets only new lines. The ring is not written to disk, so it starts empty after a restart. Control characters are replaced with spaces.

Because it is not on disk, the activity log is not a record. For a record of a failed install or update, the Suite Log, the Full Update's job log, or the tool's own journal is the place to look.

What the log system shows

8.1 Logs are kept in `/run`

The Suite Log and its flag live in `/run`, which is cleared at each reboot. A reboot therefore ends a collection and removes its file. The book should say so, and tell operators to download a collection before a reboot.

8.2 Limits protect the Pi

The rate limit, the memory guard and the 10 MB rotation keep a collection from filling the small SD card or starving the node. The book can describe them as its safety rules.

8.3 Redaction is useful, not a guarantee

The redaction hides the common secret forms and tags addresses. It does not know about every secret a log line could carry. The book should advise checking a collection before it leaves the node.

8.4 Not checked

- The viewer page's behaviour in a browser (its query and coverage views were read for their routes).
- The viewer's timeline strip (up to 360 one-minute bars), which was read only in outline.
- The redaction patterns in a live log with real secrets.
- The resume-after-restart path, which was read by name.